

Function parameter constraints are fragile

Document #: P2089R0
Date: 2020-01-20
Project: Programming Language C++ SG7
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

§ 1 Introduction

[[P1733R0](#)] introduced the idea of function parameter constraints, which was then elaborated upon and extended in [[P2049R0](#)]. The initial example demonstrating the feature at hand is:

```
double pow( double base, int iexp );  
double pow( double base, int iexp ) requires (iexp == 2); // proposed
```

But whose real motivation is to allow:

```
namespace std::meta {  
    struct class_info {  
        constexpr class_info(info x) requires is_class(x);  
    };  
}  
  
constexpr std::meta::class_info c = reflexpr(some_class); // proposed ok
```

That is, to allow a rich type hierarchy for reflection while still getting all the benefits that the monotype info API is able to provide.

The way this is intended to work, from the initial paper, is described as:

Overload resolution in C++ happens at compile time, *not* run time, so how could this ever work? Consider the call to `pow` in the following function:

```
void f(double in) {  
    in += 5.0;  
    double d = pow(in, 2);  
    // ...  
}
```

Here the compiler knows *at compile time* that the second argument to `pow` is `2` so it can theoretically make use of the overload with the parameter constraint. In what other cases does the compiler know at compile time the value of a parameter? As it turns out, we already have standardese for such an argument (or generally an expression) in C++: *constant expression*.

In short, this concepts extension will allow for parameter identifiers to appear in requires clauses and during overload resolution:

- if the argument is a *constant expression* it is evaluated as part of evaluation of the requires clause, and
- if the argument is *not* a *constant expression* the entire overload is discarded.

§ 2 Problems

I think this proposal has a few problems.

§ 2.1 Ephemerality

The fundamental problem is that whether or not an expression is a *constant expression* is an ephemeral property of an expression. It has a tendency to not last as long as you want it to. Relying on an expression being a constant expression is going to prevent a whole class of abstractions using normal programming models.

Let's just start with the pow example. We had:

```
double pow(double base, int exp); // #1
double pow(double base, int exp) requires (exp == 2); // #2

pow(3, 3); // calls #1
pow(3, 2); // calls #2
```

Cool. What if what we *really* wanted was $b^e + 1$? No problem, we just write a new overload:

```
double powp1(double base, int exp) { return pow(base, exp) + 1; }

powp1(3, 3); // calls #1
powp1(3, 2); // also calls #1
```

Right, we can't wrap, because once we get to the body we don't have constant expressions anymore. Likewise, we cannot even name the other pow:

```
auto p = pow; // always #1, no way to take a pointer to #2
```

And the *only* way to properly wrap pow is to actually manually write:

```
double powp1(double base, int exp) { return pow(base, exp) + 1; }
double powp1(double base, int exp) requires (exp == 2) { return pow(base, exp)
+ 1; }
```

Just kidding. That's still wrong! We have to *actually* write:

```
double powp1(double base, int exp) { return pow(base, exp) + 1; }
double powp1(double base, int exp) requires (exp == 2) { return pow(base, 2) +
1; }
```

Think about how we might abstract if our constraint was more involved than a simple `==`.

Let's go back to what really motivated this feature. This can work fine:

```
constexpr std::meta::class_info c = reflexpr(some_class);
```

Because `reflexpr(some_class)` is a constant expression. Indeed, even this can work fine:

```
constexpr std::meta::info i = reflexpr(some_class);
constexpr std::meta::class_info c = i;
```

Because `i` is also a constant expression. But what happens when we try to use other library features:

```
std::vector<std::meta::class_info> classes;
classes.push_back(reflexpr(some_class)); // ok
classes.emplace_back(reflexpr(some_class)); // error
```

`push_back` succeeds because it takes a `class_info` `&&`, so the conversion happens while our expression is still a constant expression. But `emplace_back` fails because it deduces its parameter to `info` `&&` and has to perform the construction of `class_info` internally, at which point our object is no longer a constant expression.

The general problem here is that the conversion has to happen *right away*, before we pass any function boundaries. If we stay as an `info` for too long, we lose all ability to make these conversions:

```
constexpr void f(std::meta::info i) {
    constexpr std::meta::class_info c = i; // ill-formed
}

f(reflexpr(some_class));
```

§ 2.1.1 Literal zero as null pointer constant

This idea is reminiscent of another language feature we have: the fact that the literal zero is a null pointer constant. But since the type of the literal zero is still `int`, this vanishes quickly:

```
int* p = 0; // ok

constexpr auto zero = 0;
int* p2 = zero; // ill-formed, even though zero is a constant expression
```

Which presents very similar problems with forwarding:

```
void f(int*);

template <typename... Ts>
void wrap_f(Ts... ts) {
    f(ts...);
}

f(0); // ok
wrap_f(0); // ill-formed
```

§ 2.1.2 Narrowing from constant expressions

There's also a similar preexisting language feature with regards to narrowing:

```
constexpr int ci = 2;
constexpr short cs{ci}; // ok

int i = 2;
short s{i}; // error: narrowing
```

But while the construction of `s` is narrowing, it is at least possible to construct `s` in a different way. This suggests that we would at least need to add a “back-up” conversion mechanism from

`meta::info` to `meta::class_info`.

§ 2.2 Type-based overload resolution

The proposal at hand introduces the notion of value-based overloading, but everything else in the language and library only ever deal with type-based overloading.

What would `constructible_from<meta::class_info, meta::info>` yield? By the rules laid out in these papers, it would yield `false`. Except sometimes, it actually is constructible - but only from specific values, and only in specific situations.

Consider:

```
template <std::meta::class_info C> struct X { };
template <auto I> requires is_class(I) struct Y { };
template <convertible_to<meta::class_info> auto I> struct Z { };

X<reflexpr(some_class)> x; // ok
Y<reflexpr(some_class)> y; // ok
Z<reflexpr(some_class)> z; // error, probably?
```

Dealing with these types properly ends up requiring their own little shadow library; we'd have our normal concepts for types and then our function concepts for reflection.

Also, what would this mean:

```
template <is_class auto I> struct Q { };
```

For normal (type-based) concepts, this means `requires is_class<decltype(I)>`. But that's ill-formed for these new function concepts, it would have to mean `requires is_class(I)`, if anything. Which means we'd have to make a choice of either not having a terse syntax for this case or having a terse syntax have different semantics from other, similar-looking terse syntax.

§ 2.3 Function parameters aren't constant expressions except when they are

Everyone trying to do something during constant evaluation will eventually try to do something to the effect of:

```
constexpr int foo(int i) {  
    // or any other code which requires i to  
    // be a constant expression  
    static_assert(i >= 0);  
    // ...  
}  
  
foo(42);
```

And be surprised that this fails, even though the function is `constexpr`, even though the argument is a constant expression. And so we have to repeat the mantra over and over that function parameters are never constant expressions. Function parameters are never constant expressions.

Except, suddenly, with this paper, they can be. But only in a `requires` clause. This adds more wrinkles into an already very-complex model that just makes it harder to understand.

§ 3 Conclusion

Function parameter constraints is a creative and interesting compromise to trying to have both a monotype and a rich class hierarchy, but it presents its own problems that neither of the original choices had - and I think it has the potential to lead to a ton more confusion.

I am not sure that these problems are solvable without much more involved language changes, so in light of wanting reflection sooner rather than later, I think we should reconsider the direction of constrained function parameters.

§ 4 References

[P1733R0] David Sankel, Daveed Vandevoorde. 2019. User-friendly and Evolution-friendly Reflection: A Compromise.

<https://wg21.link/p1733r0>

[P2049R0] Andrew Sutton, Wyatt Childers. 2020. Constraint refinement for special-cased functions.

<https://wg21.link/p2049r0>